CPU Scheduling 1

Week 6

Operating Systems Ch 5 (Sections 5.1 – 5.3)

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

This Week's Learning Objectives

1. Understand the CPU-I/O Burst Cycle and explain the need for scheduling
2. Distinguish between Preemptive vs Nonpreemptive scheduling
3. Explain the role of the Dispatcher and Dispatch Latency
4. List and explain the 5 Scheduling Criteria
5. Understand FCFS, SJF, SRTF, RR algorithms and draw Gantt Charts
6. Perform CPU burst prediction using Exponential Averaging

Part 1

Basic Concepts (5.1)

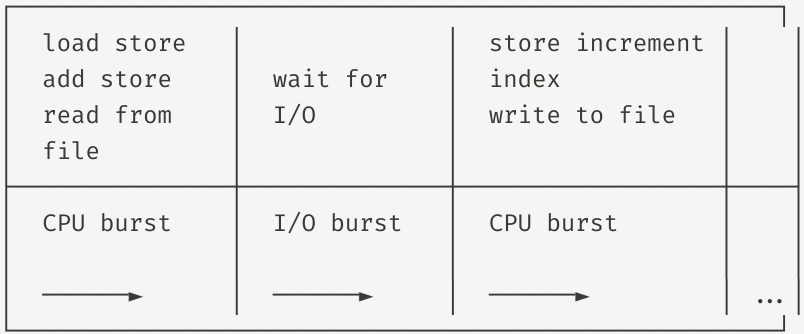
Multiprogramming and CPU Scheduling

Key Idea: Keep the CPU busy at all times to **maximize utilization**

- In a single CPU core system, only one process can run at a time
- While a process waits for I/O, the CPU is **idle** – wasted
- **Multiprogramming**: Keep multiple processes in memory; when one waits, allocate the CPU to another process
- CPU scheduling is one of the **most fundamental functions** of an operating system

CPU-I/O Burst Cycle

Process execution consists of a repeating cycle of **CPU bursts** and **I/O bursts**



- **CPU burst**: The interval during which the CPU executes instructions
- **I/O burst**: The interval during which the process waits for I/O completion
- In the final CPU burst, the process requests **termination via a system call**

CPU-I/O Burst Cycle – Detailed Flow

Expressing a process's lifecycle in code:

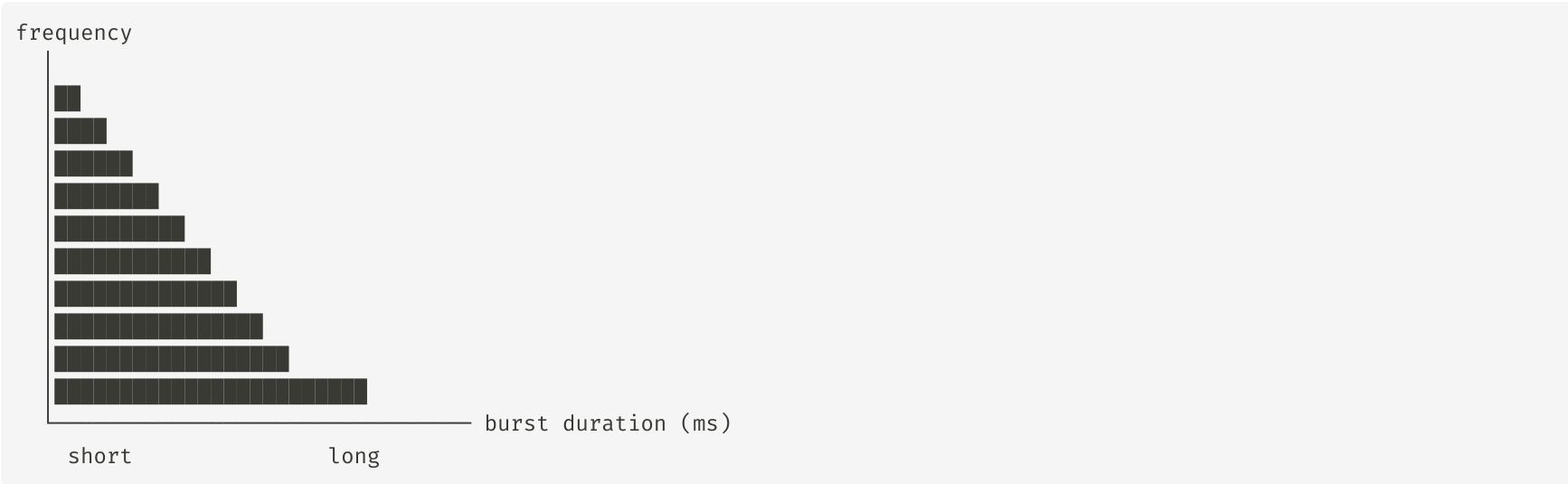
```
// Process execution pattern (pseudocode)
while (process_not_terminated) {
    // ═══ CPU Burst ═══
    execute_instructions();   // load, store, add, ...

    // ═══ I/O Burst ═══
    request_io();             // read from file, write to file, ...
    wait_for_io_completion(); // blocking
}
terminate();                  // Terminates after the last CPU burst
```

- All processes follow this pattern, but the **burst length and frequency** differ
- This difference is the key basis for choosing scheduling algorithms

CPU Burst Distribution (Figure 5.2)

The distribution of CPU burst lengths follows an **exponential or hyperexponential** decay pattern



- **Short CPU bursts** are very frequent – most bursts are short
- **Long CPU bursts** occur rarely

I/O-bound vs CPU-bound Process

Process classification based on CPU burst distribution

Category	I/O-bound Process	CPU-bound Process
CPU burst	Many **short** bursts	Few **long** bursts
I/O burst	Frequent I/O requests	Rare I/O requests
Examples	Web server, database	Scientific computation, video encoding
Scheduling needs	Fast response time	Long CPU allocation time

- **Scheduling algorithms** must efficiently handle both types
- This distribution characteristic forms the theoretical basis for algorithms like SJF and RR

CPU Scheduler (Short-term Scheduler)

When the CPU becomes idle, the **ready queue** is used to select the next process to run

```
new process ──►  [ Ready Queue ]  ──► CPU Scheduler ──► CPU
                                      selection algorithm
                       ▲
                       |
                I/O completion, etc.
```

- **CPU Scheduler** = Short-term Scheduler
- The ready queue does not have to be FIFO
  - Can be implemented as a FIFO queue, **priority queue**, tree, linked list, etc.
- Records in the queue are typically **PCBs (Process Control Blocks)**
- The internal order of the queue is determined by the scheduling algorithm

When Scheduling Occurs – 4 Circumstances

4 circumstances where CPU scheduling decisions are needed:

No.	State Transition	Example	Choice
1	Running → Waiting	I/O request, wait() call	No choice
2	Running → Ready	Interrupt occurs (timer, etc.)	**Choice exists**
3	Waiting → Ready	I/O completion	**Choice exists**
4	Terminates	Process terminates	No choice

- Circumstances **1, 4**: A new process must be selected (no choice)
- Circumstances **2, 3**: Depends on the scheduling scheme (choice exists)

Nonpreemptive vs Preemptive Scheduling

Category	Nonpreemptive	Preemptive
Scheduling points	Only at **1, 4**	At **1, 2, 3, 4** (all)
CPU release	Process **voluntarily** releases	OS can **forcibly** take away
Also known as	Cooperative scheduling	-
Usage	Early OS	**Most modern OS**

- **Nonpreemptive**: Process voluntarily releases the CPU (termination or I/O wait)
- **Preemptive**: OS can forcibly reclaim the CPU via timer interrupt, etc.
- Windows, macOS, Linux, UNIX – all use **preemptive** scheduling

Issues with Preemptive Scheduling

Preemptive scheduling is powerful but has **caveats**:

1. Race Condition

- Preemption while modifying shared data → another process reads inconsistent data
- → **Synchronization mechanisms** (mutex, semaphore, etc.) are needed (covered in detail in Chapter 6)

2. Preemption in Kernel Mode

- Preemption during kernel data structure modification within a system call → kernel inconsistency
- **Nonpreemptive kernel**: Defer context switch until system call completes (simple but unsuitable for real-time)
- **Preemptive kernel**: Protect kernel data with mutex locks, etc. (most modern OS)

3. Interrupt Handling

- Interrupts can occur at any time → disable interrupts in critical sections

Dispatcher

Dispatcher: The module that actually hands over CPU control to the process selected by the Scheduler

```
┌──────────────┐   save state      ┌──────────────┐
│ Process P0   │──── into PCB0 ──►│ Process P1   │
│ executing    │                   │              │
│              │                   │ restore      │
│              │  dispatch         │ state from   │
│              │  latency          │ PCB1         │
│              ◄──────────────────│ executing    │
└──────────────┘                   └──────────────┘
```

Three roles of the Dispatcher:

1. Perform **context switch** (save current process state → restore new process state)
2. Switch to **user mode**
3. Jump to the appropriate location (**PC**) of the new process

Dispatch Latency

Dispatch Latency = The time it takes for the Dispatcher to stop one process and start running another

- Occurs on every context switch, so it should be **as short as possible**
- Checking context switch counts on Linux:

```
# System-wide context switch count (1-second interval, 3 times)
vmstat 1 3

# Context switch count for a specific process
cat /proc/<PID>/status | grep ctxt
# voluntary_ctxt_switches:    150   (voluntary: I/O wait, etc.)
# nonvoluntary_ctxt_switches: 8     (involuntary: time slice expired, etc.)
```

- **Voluntary context switch**: Process voluntarily releases the CPU due to lack of resources
- **Nonvoluntary context switch**: OS forces a switch due to time slice expiration, etc.

Part 2

Scheduling Criteria (5.2)

Scheduling Criteria – Overview

Five criteria for **comparing** and **evaluating** scheduling algorithms:

Criterion	Description	Optimization Direction
CPU Utilization	CPU usage rate (%)	**Maximize**
Throughput	Number of processes completed per unit time	**Maximize**
Turnaround Time	Total time from submission to completion	**Minimize**
Waiting Time	Total time spent waiting in the ready queue	**Minimize**
Response Time	Time from submission to first response	**Minimize**

CPU Utilization & Throughput

CPU Utilization

- The proportion of time the CPU is performing useful work
- Theoretical range: 0% to 100%
- Real systems: around **40%** under light load, **90%** under heavy load
- Can be checked with the `top` command on Linux

Throughput

- Number of processes completed per unit time
- Long processes: one every few seconds
- Short transactions: dozens per second
- The goal is to **maximize** both criteria
- They can be in a trade-off relationship (context switch overhead)

Turnaround Time & Waiting Time

Turnaround Time

- Total time from process **submission** to **completion**
- = Sum of ready queue wait + CPU execution + I/O wait time

```
Turnaround Time = Completion Time - Arrival Time
```

Waiting Time

- Total time spent **waiting** in the ready queue
- The only time that the scheduling algorithm **directly affects**
- CPU execution time and I/O time are independent of the algorithm

```
Waiting Time = Turnaround Time - Burst Time
             (for a single burst, with no I/O)
```

Response Time

Response Time

- Time from process **submission** to the moment the **first response is produced**
- The most important criterion in interactive systems

```
Response Time = First Run Time - Arrival Time
```

Turnaround Time vs Response Time

Comparison	Turnaround Time	Response Time
Measured interval	Submission to **completion**	Submission to **first response**
Important for	Batch system	**Interactive** system
Example	30 seconds to complete the job	0.5 seconds to first output

- In interactive systems, **minimizing the variance** of response time is important

Scheduling Criteria – Summary

```
┌──────────────────────────────────────────────────────────┐
│                      Maximize                            │
│                                                          │
│  ┌──────────────────┐   ┌──────────────────┐            │
│  │ CPU Utilization  │   │ Throughput       │            │
│  │                  │   │                  │            │
│  └──────────────────┘   └──────────────────┘            │
├──────────────────────────────────────────────────────────┤
│                      Minimize                            │
│                                                          │
│  ┌──────────────────┐   ┌──────────────────┐            │
│  │ Turnaround Time  │   │ Waiting Time     │            │
│  │                  │   │                  │            │
│  └──────────────────┘   └──────────────────┘            │
│                                                          │
│  ┌──────────────────┐                                    │
│  │ Response Time    │                                    │
│  │                  │                                    │
│  └──────────────────┘                                    │
└──────────────────────────────────────────────────────────┘
```

- In most cases, the **average** is optimized
- In some cases, the **minimum** or **maximum** is optimized (e.g., minimizing the maximum response time)

Part 3

FCFS Scheduling (5.3.1)

FCFS (First-Come, First-Served)

The **simplest** CPU scheduling algorithm: execute in **arrival order**

- Implemented with a **FIFO queue**
- When a process arrives at the ready queue, it is added to the **tail**
- When the CPU is free, it is allocated to the process at the **head**
- **Nonpreemptive** scheduling

```
Ready Queue (FIFO):

  head ──► [ P1 ] ──► [ P2 ] ──► [ P3 ] ◄── tail
             │
             ▼
          CPU allocation
```

FCFS – Example 1 (Arrival Order: P1, P2, P3)

Process arrival order: P1 → P2 → P3 (all arrive at time 0)

Process	Burst Time
P1	24 ms
P2	3 ms
P3	3 ms

```
Gantt Chart:

| P1                    | P2 | P3 |

0                       24   27   30
```

Process	Waiting Time	Turnaround Time

FCFS – Example 2 (Arrival Order: P2, P3, P1)

Same processes but different arrival order: P2 → P3 → P1

```
Gantt Chart:

| P2 | P3 |            P1            |

0     3     6                       30
```

Process	Waiting Time	Turnaround Time
P1	6	30
P2	0	3
P3	3	6
Average	**(6+0+3)/3 = 3**	**(30+3+6)/3 = 13**

- Average Waiting Time: **17 → 3** (dramatically reduced just by changing the order!)

FCFS – Convoy Effect

Convoy Effect: A phenomenon where short processes queue up behind a single process with a long CPU burst

```
Scenario: 1 CPU-bound process + multiple I/O-bound processes

Time →  ═══════════════════════════════════════════════════════►

CPU:  [ CPU-bound (long burst) .......................][ I/O-1 ][ I/O-2 ][ I/O-3 ]
I/O:  [     idle (I/O device sitting idle)            ][ busy  ][ busy  ][ busy  ]
                                                        ▲
                                                        |
                                                I/O-bound processes
                                                waiting behind CPU-bound
```

- The CPU-bound process monopolizes the CPU → I/O-bound processes wait in the ready queue
- Meanwhile, the **I/O device is idle** → both CPU and I/O **utilization drops**
- **Unsuitable for interactive systems**: a single process can monopolize the CPU for a long time

FCFS – Characteristics Summary

Characteristic	Description
Type	**Nonpreemptive**
Implementation	FIFO queue – very simple
Average Waiting Time	Generally **long** (not optimal)
Convoy Effect	**Occurs** (CPU-bound blocks I/O-bound)
Interactive suitability	**Unsuitable** (cannot distribute CPU at regular intervals)
Advantage	Simple to implement, easy to understand
Disadvantage	Performance heavily depends on arrival order

Part 4

SJF Scheduling (5.3.2)

SJF (Shortest-Job-First)

Select the process with the **shortest next CPU burst** first

- More accurate name: **Shortest-Next-CPU-Burst** Algorithm
- If burst lengths are equal → tie-breaking with **FCFS**
- Both Nonpreemptive and Preemptive versions exist

Key property: Guarantees **minimum average waiting time** for a given set of processes – **Optimal**

Fundamental problem: The next CPU burst length **cannot be known exactly**

SJF – Proof of Optimality (Intuitive Understanding)

Why is SJF optimal?

```
Case 1: Long first (burst: 10, 3)

 ┌──────────┬──┐
 │  P1(10)  │P2│     P1 wait=0, P2 wait=10  → average = 5
 └──────────┴──┘
0          10  13

Case 2: Short first (burst: 3, 10)  ← SJF

 ┌──┬──────────┐
 │P2│  P1(10)  │     P2 wait=0, P1 wait=3   → average = 1.5
 └──┴──────────┘
0   3          13
```

- Moving the **shorter process** forward: its waiting time **decreases significantly**
- The **longer process's** waiting time **increases slightly**
- Result: The overall average waiting time **decreases**
- Generalizing this shows that SJF is **provably optimal**

SJF – Example (Nonpreemptive)

All processes arrive at time 0 (nonpreemptive SJF)

Process	Burst Time
P1	6 ms
P2	8 ms
P3	7 ms
P4	3 ms

```
Gantt Chart (SJF order: P4 → P1 → P3 → P2):

| P4 | P1 | P3 | P2 |

0     3        9         16         24
```

The Problem with SJF – Predicting the Next Burst Length

SJF is **optimal** but impossible to implement in practice – because the **next CPU burst length cannot be known**

Solution: **Prediction** based on past burst history

- Past CPU bursts tend to repeat with similar lengths
- Use **Exponential Averaging** for prediction

Exponential Averaging – Formula

Formula for predicting the next CPU burst:

```
τ(n+1) = α · t(n) + (1 - α) · τ(n)
```

Symbol	Meaning
`t(n)`	The n-th **actual** CPU burst length (most recent observed value)
`τ(n)`	The n-th **predicted value** (previous prediction)
`τ(n+1)`	The next CPU burst **predicted value**
`α`	Weight ($0 \leq \alpha \leq 1$)

Characteristics depending on α:

Exponential Averaging – Expansion

Expanding the formula by repeated substitution:

```
τ(n+1) = α · t(n)
       + (1-α) · α · t(n-1)
       + (1-α)² · α · t(n-2)
       + ...
       + (1-α)ʲ · α · t(n-j)
       + ...
       + (1-α)^(n+1) · τ(0)
```

- If α < 1, then **(1 − α)** is also < 1 → weights **decrease exponentially** going further into the past
- The most recent observed value has the **greatest influence** on the prediction
- This is the origin of the name **"exponential averaging"**: the weights decay exponentially

Exponential Averaging – Calculation Example

Starting with $\alpha = 0.5$, $\tau(0) = 10$

n	Actual burst t(n)	Predicted τ(n)	Calculation
0	6	**10**	Initial value
1	4	**8**	$0.5\times6 + 0.5\times10 = 8$
2	6	**6**	$0.5\times4 + 0.5\times8 = 6$
3	4	**6**	$0.5\times6 + 0.5\times6 = 6$
4	13	**5**	$0.5\times4 + 0.5\times6 = 5$
5	13	**9**	$0.5\times13 + 0.5\times5 = 9$
6	13	**11**	$0.5\times13 + 0.5\times9 = 11$

Exponential Averaging – Comparing α Values

Prediction differences for varying α on the same burst sequence ($\tau_0 = 10$, bursts = [6, 4, 6])

```
                    α = 0.2        α = 0.5        α = 0.8
τ(0) = 10          10             10             10
t(0) = 6
τ(1)               9.2            8.0            6.8
t(1) = 4
τ(2)               8.16           6.0            4.56
t(2) = 6
τ(3)               7.73           6.0            5.71
```

- **Smaller α**: Predicted value changes **slowly** (more dependent on history)
- **Larger α**: Predicted value **reacts quickly** to recent values
- In practice, values around **α = 0.5** are commonly used

Part 5

SRTF – Preemptive SJF (5.3.2)

SRTF (Shortest-Remaining-Time-First)

Preemptive version of SJF = SRTF

How it works:

- **Comparison is performed each time** a new process arrives
- If the new process's burst < the current process's **remaining burst → preempt (switch)**
- Otherwise, the current process continues execution

Comparison	SJF (Nonpreemptive)	SRTF (Preemptive)
Comparison point	Only when CPU is free	**Each time** a new process arrives
Comparison target	Total burst length	**Remaining** burst length
Preemption	None	Yes

SRTF – Example Data

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Let us examine the decision at each time step.

SRTF – Step-by-Step Execution

Time 0: Only P1 has arrived → P1 starts executing (remaining: 8)

Time 1: P2 arrives (burst=4) vs P1 (remaining=7) → 4 < 7 → **Preempt P1, run P2**

Time 2: P3 arrives (burst=9) vs P2 (remaining=3) → 9 > 3 → **P2 continues**

Time 3: P4 arrives (burst=5) vs P2 (remaining=2) → 5 > 2 → **P2 continues**

Time 5: P2 completes → ready: P1(remaining=7), P3(9), P4(5) → **Select P4** (5 is minimum)

Time 10: P4 completes → ready: P1(remaining=7), P3(9) → **Select P1**

Time 17: P1 completes → ready: P3(9) → **Select P3**

Time 26: P3 completes → All processes finished

SRTF – Gantt Chart and Calculation

```
Gantt Chart:

| P1 | P2 | P4 | P1 | P3 |

0    1         5           10              17                26
```

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	8	17	17-0 = 17	17-8 = **9**
P2	1	4	5	5-1 = 4	4-4 = **0**
P3	2	9	26	26-2 = 24	24-9 = **15**
P4	3	5	10	10-3 = 7	7-5 = **2**

```
Average Waiting Time    = (9 + 0 + 15 + 2) / 4 = 26/4 = 6.5 ms
Average Turnaround Time = (17 + 4 + 24 + 7) / 4 = 52/4 = 13.0 ms
```

SJF vs SRTF – Same Data Comparison

Comparing Nonpreemptive SJF with the same data (P1:0,8 / P2:1,4 / P3:2,9 / P4:3,5)

```
Nonpreemptive SJF:
| P1 | P2 | P4 | P3 |
0        8    12   17        26
```

Algorithm	P1 wait	P2 wait	P3 wait	P4 wait	**Average**
SJF	0	7	15	9	**7.75**
SRTF	9	0	15	2	**6.5**

- SRTF is **superior** in average waiting time (6.5 < 7.75)
- Preemption allows shorter processes to be handled faster

Part 6

Round-Robin Scheduling (5.3.3)

Round-Robin (RR)

Each process is allocated a **fixed time quantum (q)** and executed in rotation

How it works:

1. Manage the ready queue as a **circular queue**
2. Each process uses the CPU for at most **q time units**
3. If still running after q time → **preempt**, move to the **end** of the ready queue
4. If the CPU burst is shorter than q → voluntarily release the CPU

- **Preemptive** scheduling
- Essentially FCFS with **preemption** added

Round-Robin — Fairness Guarantee

With n processes and time quantum = q:

```
Each process receives 1/n of the CPU time (in chunks of at most q)

Maximum wait time: (n - 1) × q

Example: n = 5, q = 20ms
→ Each process gets up to 20ms of CPU every 100ms
→ Waits at most 80ms before getting the CPU again
```

- No process waits more than **(n-1) × q**
- Favorable for guaranteeing **response time**
- **Fair** CPU allocation to all processes

RR – Example (q = 4ms)

All processes arrive at time 0, time quantum = 4ms

Process	Burst Time
P1	24 ms
P2	3 ms
P3	3 ms

```
Gantt Chart:

| P1 | P2 | P3 | P1 | P1 | P1 | P1 | P1 | P1 |

0      4     7    10     14     18     22     26     30
```

- P1: Runs 4ms then preempted → P2: 3ms (< q) voluntarily returns → P3: 3ms returns → P1 repeats

RR – Example Calculation

Process	Execution Intervals	Waiting Time	Turnaround
P1	0-4, 10-14, 14-18, 18-22, 22-26, 26-30	10-4 = **6**	**30**
P2	4-7	**4**	**7**
P3	7-10	**7**	**10**

```
P1 Waiting = (10 - 4) = 6     (waited during 4~10)
P2 Waiting = 4 - 0 = 4        (waited during 0~4)
P3 Waiting = 7 - 0 = 7        (waited during 0~7)

Average Waiting Time = (6 + 4 + 7) / 3 = 17/3 ≈ 5.67 ms
```

- FCFS (order P1,P2,P3): average waiting = **17 ms**
- RR (q=4): average waiting = **5.67 ms** → significant improvement!

RR – When the Time Quantum Is Too Large

If q is very large (larger than all bursts):

```
When q = 100 (all bursts < 100):

 ┌──────────────────────────┬──────┬──────┐
 │          P1(24)          │P2(3) │P3(3) │
 └──────────────────────────┴──────┴──────┘

0                           24    27    30

→ Behaves identically to FCFS!
→ Preemption never occurs
```

- If the time quantum is **larger than all CPU bursts**, RR = FCFS
- The response time improvement effect disappears

RR — When the Time Quantum Is Too Small

If q is very small (e.g., q = 1ms):

```
1 process, burst = 10:

q = 12:  [    P1(10)    ]               context switches: 0
         0              10

q = 6:   [  P1  ][  P1  ]               context switches: 1
         0      6      10+1

q = 1:   [P][P][P][P][P][P][P][P][P][P] context switches: 9
         0  1  2  3  4  5  6  7  8  9  10+9
```

- Context switch count **increases dramatically** → overhead grows
- If context switch time exceeds 10% of the quantum → **inefficient**
- Real systems: context switch ~ **10 us**, quantum ~ **10–100 ms**

RR – Time Quantum and Turnaround Time

Effect of q on turnaround time (3 processes, each burst = 10)

```
q = 1:  P1,P2,P3,P1,P2,P3, ...  → average turnaround = 29
q = 10: P1,P2,P3              → average turnaround = 20
```

Rule of Thumb:

- Ideally, **80%** of CPU bursts should finish within the time quantum
- If q is too small: turnaround worsens due to context switch overhead
- If q is too large: behaves like FCFS, worsening response time
- Choosing an appropriate q is **critical**

RR – Detailed Example (4 Processes)

q = 3ms, all processes arrive at time 0

Process	Burst Time
P1	8 ms
P2	5 ms
P3	3 ms
P4	6 ms

```
Gantt Chart (q = 3):

| P1 | P2 | P3 | P4 | P1 | P2 | P4 | P1 |

0     3     6     9     12    15    17    20    22
```

- P3 has burst=3=q → completes in one turn

RR – Detailed Example Calculation

Process	Burst	Completion	Turnaround	Waiting
P1	8	22	22	**14**
P2	5	17	17	**12**
P3	3	9	9	**6**
P4	6	20	20	**14**

```
Average Waiting Time    = (14 + 12 + 6 + 14) / 4 = 46/4 = 11.5 ms
Average Turnaround Time = (22 + 17 + 9 + 20) / 4 = 68/4 = 17.0 ms
```

- RR has a longer average turnaround than SJF but **superior response time**
- All processes use the CPU relatively **fairly**

Algorithm Comparison – 4 Algorithms on the Same Data

Data: P1(0,24), P2(0,3), P3(0,3) – all arrive at time 0

Algorithm	P1 wait	P2 wait	P3 wait	**Avg WT**	Note
FCFS (P1→P2→P3)	0	24	27	**17.0**	Worst case
FCFS (P2→P3→P1)	6	0	3	**3.0**	Order-dependent
SJF	6	0	3	**3.0**	Optimal
RR (q=4)	6	4	7	**5.67**	Fairness

- **SJF** is optimal in average waiting time
- **RR** is advantageous in terms of response time
- **FCFS** shows high performance variability depending on order

Algorithm Comparison – Comprehensive Characteristics

Characteristic	FCFS	SJF	SRTF	RR
Preemption	No	No	**Yes**	**Yes**
Optimal WT	No	**Yes**	**Yes** (preemptive)	No
Starvation	No	Possible	Possible	**No**
Response Time	Poor	Moderate	Moderate	**Excellent**
Implementation complexity	Very simple	Burst prediction needed	Burst prediction needed	Timer needed
Convoy Effect	**Occurs**	None	None	None

Lab

Scheduling Algorithm Simulator

Lab – Implementing a Scheduling Simulator

Implement a scheduling simulator for FCFS, SJF, SRTF, and RR

Input

- Process list: arrival time and burst time for each process
- Time quantum for RR

Output Requirements

1. **Gantt Chart**: Visual display of execution order for each algorithm
2. **Statistics Calculation**: Waiting time, turnaround time for each process, and averages

Lab – Data Structure

```python
class Process:
    def __init__(self, pid, arrival, burst):
        self.pid = pid
        self.arrival = arrival
        self.burst = burst
        self.remaining = burst    # Used in SRTF, RR
        self.completion = 0
        self.waiting = 0
        self.turnaround = 0
        self.first_run = -1       # For response time calculation

def print_gantt(timeline):
    """timeline: [(pid, start, end), ...]"""
    header = "|"
    footer = ""
    for pid, start, end in timeline:
        width = max(len(f" P{pid} "), end - start)
        header += f" P{pid} ".center(width) + "|"
    print(header)
    # Time display
    print("0", end="")
    for pid, start, end in timeline:
        print(f"{'':>{end - start + 2}}{end}", end="")
    print()
```

Lab – FCFS Implementation

```python
def fcfs(processes):
    """First-Come, First-Served Scheduling"""
    # Sort by arrival order
    procs = sorted(processes, key=lambda p: p.arrival)
    time = 0
    timeline = []

    for p in procs:
        if time < p.arrival:
            time = p.arrival  # CPU idle

        start = time
        time += p.burst
        p.completion = time
        p.turnaround = p.completion - p.arrival
        p.waiting = p.turnaround - p.burst
        timeline.append((p.pid, start, time))

    return timeline

# Test
procs = [Process(1,0,24), Process(2,0,3), Process(3,0,3)]
fcfs(procs)
# → Average Waiting Time = 17.0
```

Lab – SJF Implementation (Nonpreemptive)

```python
def sjf(processes):
    """Shortest-Job-First (Nonpreemptive)"""
    procs = [p for p in processes]  # Copy
    n = len(procs)
    completed = 0
    time = 0
    done = [False] * n
    timeline = []

    while completed < n:
        # Select the arrived process with the shortest burst
        candidates = [(i, procs[i]) for i in range(n)
                      if not done[i] and procs[i].arrival ≤ time]

        if not candidates:
            time += 1
            continue

        idx, p = min(candidates, key=lambda x: x[1].burst)
        start = time
        time += p.burst
        p.completion = time
        p.turnaround = p.completion - p.arrival
        p.waiting = p.turnaround - p.burst
```

Lab – RR Implementation

```python
from collections import deque

def round_robin(processes, quantum):
    """Round-Robin Scheduling"""
    procs = sorted(processes, key=lambda p: p.arrival)
    for p in procs:
        p.remaining = p.burst

    queue = deque()
    time = 0
    idx = 0
    timeline = []

    # Add processes arriving at time 0
    while idx < len(procs) and procs[idx].arrival ≤ time:
        queue.append(procs[idx])
        idx += 1

    while queue:
        p = queue.popleft()
        run_time = min(quantum, p.remaining)
        start = time
        time += run_time
        p.remaining -= run_time
```

Lab – Testing and Comparing Results

```python
# Test data
data = [(1, 0, 10), (2, 1, 5), (3, 2, 8), (4, 3, 6)]

print("═══ FCFS ═══")
procs = [Process(*d) for d in data]
fcfs(procs)
for p in procs:
    print(f"P{p.pid}: WT={p.waiting}, TT={p.turnaround}")

print("\n═══ SJF ═══")
procs = [Process(*d) for d in data]
sjf(procs)
for p in procs:
    print(f"P{p.pid}: WT={p.waiting}, TT={p.turnaround}")

print("\n═══ RR (q=4) ═══")
procs = [Process(*d) for d in data]
round_robin(procs, 4)
for p in procs:
    print(f"P{p.pid}: WT={p.waiting}, TT={p.turnaround}")
```

Test data: P1(0,10), P2(1,5), P3(2,8), P4(3,6) – compare results across all 3 algorithms.

Summary

Key concepts this week

Topic	Key Points
CPU-I/O Burst	Processes **alternate** between CPU bursts and I/O bursts
CPU Scheduler	**Selects** the next process from the ready queue (Short-term Scheduler)
Preemptive vs Nonpreemptive	Most modern OS use **preemptive** (all 4 circumstances)
Dispatcher	Performs context switch, switches to user mode, jumps to PC
Scheduling Criteria	CPU Util, Throughput ↑ / TT, WT, RT ↓
FCFS	Simple but causes **Convoy Effect**
SJF	Optimal but burst **prediction impossible** (Exponential Averaging)

Next Week

Week 7: CPU Scheduling 2 (Ch 5, Sections 5.4–5.7)

- **Priority Scheduling** and Starvation / Aging
- **Multilevel Queue** / Multilevel Feedback Queue
- **Multi-Processor Scheduling** (SMP, Processor Affinity, Load Balancing)
- **Real-Time Scheduling** (Rate-Monotonic, EDF)
- **Linux CFS**, Windows Scheduling
- Algorithm Evaluation